CMPE 211 - DATA STRUCTURES AND ALGORITHMS COURSE PROJECT

YAĞIZ EFE ÇİÇEKÇİSOY - 124200053

BATUHAN TÜRKKAN – 124200129

Abstract

In this project, we extended our knowledge of hash tables beyond separate chaining (buckets) and linear probing.

As instructed in the pdf document, we chose an advanced methodology to boost the efficiency of the classic hash-table & hash-function: "Double Hashing"

After searching for the most optimized implementation techniques, we ran large-scale tests and tried multiple workloads to simulate and evaluate the general performance of the updated and enhanced version of our "Hash Decision".

As emphasized in the project, we first worked on the implementation, then proceeded with evaluating the outpust & test results. As for the bonus, we also analyzed the edge-cases regarding our choice of advanced hash techniques.

In the following part of the report, you can find the detailed analysis procedure of this study and the answers to the questions asked.

Questions and Answers:

1. Research the chosen technique and summarize its main idea.

Hash tables are one of the most profound data structures that provide efficient average-case insertion, search and deletion operations. Although they can effectively handle and accommodate a large amount of data systematically, their overall ability and performance is generally determined by how they handle collisions; that might naturally occur. When multiple keys are mapped to the same index, there are variety of advanced optimization techniques we can use, here in this project we used "Double Hashing":

Double Hashing is an advanced "open-addressing" technique that lightens the clustering problems that naturally arise in linear probing. The main idea is to use two independent hash functions.

The invariants:

1. *The second hash function is only used when there occurs a collision after using the first hash function, meaning that if the index that is obtained from the first hash function is empty, it is valid and used.*
2. *If the collision continues, then the index is again updated using the second hash function.*
3. *The second hash function is responsible for determining the step size to be used to probe an alternative address when such a collision occurs.*
4. *The second hash function must never be zero and should be relatively prime to the table size (and not very similar to the first hash function.)*

This guarantees that as long as there is an empty address, a place to store will be found and consequently there won't be infinite loops.

The main problem of the linear probing is that we might have to linearly go over all the neighboring addresses to find an empty space. This leads to contiguous blocks of linear searches. Although linear probing is relatively more cache-friendly compared to “double hashing”, still going over long intervals makes it much more costly and slower.

2. Implement the technique from scratch (no library hash maps).
Implementation can be viewed from the code file.

3. Support the standard operations:

put(key,value): This operation prevents collisions by calculating and updating the collision-causing index using the two hash functions, ensuring that each key follows a distinct probe sequence..

get(key): The get operation again uses the hash functions in the same sequence to be able to retrieve the value matching with the key index.

remove(key): This function marks the corresponding address as “DELETED” but does not actually delete. This guarantees that the probe sequence for the other keys will not also be broken. So we can say that the memory is directly parallel to the number of inputs since the removal does not actually delete the corresponding element.

contains(key): Used mainly to understand if a key exists or not.

size(): Returns the number of entries in the table, independent of the deleted slots.

4- Compare your implementation against a baseline (linear probing or separate chaining).

Our baseline choice was linear probing due to the fact that double hashing is similar to an additional mode that solves the collision problem of it.

Collision Resolution Difference: *While linear probing solves collisions with incrementing the index by 1, namely the adjacent addresses, double hashing provides better distribution and is usually more likely to find an empty slot since it wouldn't have to linear search until an empty address as it might directly land in an empty space using the second hash function.*

Clustering Effect: *Linear probing is very open to be affected by primary clustering, on the other hand double hashing generally separates and distributes the keys successfully.*

Performance Measurement: *In our tests, we used total probe count to measure and show the number of tries for an empty cell.*

Overall Result: *Double hashing obviously outperforms the linear probing, especially at high load factors and under workloads that encourage collisions.*

Experimental Evaluation (Required):

RESULTS UNDER THREE DIFFERENT LOAD FACTORS:

LoadFactor	*LinearProbing*	*DoubleHashing*
0,5	*20144*	*17086*
0,8	*123231*	*55557*
0,9	*459926*	*112181*

RESULTS UNDER THREE DIFFERENT WORKLOADS:

=== WORKLOAD A: Random Keys ===
Linear avg probes: 5
Double avg probes: 2
Avg time per op (ns): 547
Approx memory used (bytes): 1015328

=== WORKLOAD B: Clustered Keys ===
Linear avg probes: 4503
Double avg probes: 1.9998889629136132
Avg time per op (ns): 27313
Approx memory used (bytes): 1037192

=== WORKLOAD C: Mixed Insert/Delete/Search ===
Linear avg probes: 1127
Double avg probes: 7
Avg time per op (ns): 7117
Approx memory used (bytes): -1870368

Note that the memory is negative in Workload C, this is because the Java Garbage Collector is autonomously operating. But still, the approximate result is totally correct.

Bonus: Edge Cases and Failure Modes

Demonstrate and explain a known trade-off or failure mode.

One of the most significant failure modes of open addressing is clustering, where probe sequences trigger long probe chains and rapid performance issues. This effect is most harmful in linear probing due to primary clustering, especially at high load factors or under problematic key distributions.

Double hashing prevents clustering by using key-dependent probe sequences, it decreases probe counts effectively. Our experiments demonstrate that while linear probing breaks down under clustering-heavy workloads, double hashing remains robust.This proves the trade-off between simplicity and durability toward collision.

The outputs and the analysis of the hashing algorithm under different workloads and load factors prove and demonstrate that under most of the circumstances, “double hashing is far more superior to the linear probing”. Therefore, they can again be used to understand and prove this fact:

RESULTS UNDER THREE DIFFERENT LOAD FACTORS:

LoadFactor	***LinearProbing***	***DoubleHashing***
0,5	***20144***	***17086***
0,8	***123231***	***55557***
0,9	***459926***	***112181***

RESULTS UNDER THREE DIFFERENT WORKLOADS:

=== WORKLOAD A: Random Keys ===
Linear avg probes: 5
Double avg probes: 2
Avg time per op (ns): 547
Approx memory used (bytes): 1015328

=== WORKLOAD B: Clustered Keys ===
Linear avg probes: 4503
Double avg probes: 1.9998889629136132
Avg time per op (ns): 27313
Approx memory used (bytes): 1037192

=== WORKLOAD C: Mixed Insert/Delete/Search ===
Linear avg probes: 1127
Double avg probes: 7
Avg time per op (ns): 7117
Approx memory used (bytes): -1870368

Discussion and interpretation of findings.

The results indicate that double hashing outperforms linear probing by reducing clustering and probe counts, especially at high load factors. Linear probing degrades quickly under collision-heavy workloads, whereas double hashing maintains stable performance. These findings confirm the effectiveness of double hashing in mitigating clustering in open addressing.